CYBERSECURITY PORTFOLIO | AUTOMATION & ACCESS CONTROL

Update a File Through a Python Algorithm: Network Access Control

Developer / Analyst: Syed Yasir Shah, Cybersecurity Analyst

Target Asset: allow_list.txt (Restricted Healthcare Subnet)

Environment / Language: Python 3.x (File I/O, Lists, Strings, Loops)

Access Control Mechanism: Dynamic IP Allow-Listing & Revocation

1. Project Description

At my organization, access to restricted medical content—specifically electronic personal health records (ePHI)—is tightly controlled using an allow list of approved employee IP addresses. The "allow_list.txt" file identifies the specific IP addresses permitted to sign into the restricted healthcare subnetwork. A separate list, remove_list, identifies employee IP addresses that should no longer have access to this sensitive data due to role reassignments, departmental transfers, or contract conclusions.

To eliminate the high operational overhead and human error associated with manually editing network access files, I developed an automated Python algorithm. The algorithm systematically parses "allow_list.txt", cross-references active IPs against the revocation entries in remove_list, removes unauthorized addresses without throwing runtime errors, and rewrites the sanitized access list back to disk.

2. Algorithm Development & Step-by-Step Implementation

2.1 Open the File That Contains the Allow List

For the first part of the algorithm, I targeted the "allow_list.txt" file. I assigned the file name as a string to the import_file variable and utilized a Python with statement to open the file securely in read mode ("r").

```
Step 1: Open 'allow_list.txt' with open()

# Assign file name and open in read mode
import_file = "allow_list.txt"

with open(import_file, "r") as file:
    # File is now opened and managed in memory
```

In my algorithm, the with statement is used in combination with the open() function in read mode to open the allow list file for reading. The purpose of opening the file is to allow program access to the IP addresses stored within. The with keyword manages system resources automatically by ensuring the file is safely closed immediately after the code block exits, preventing file handle leaks. In the code open(import_file, "r") as file:, the open() function takes two parameters: the file path string (import_file) and the access mode ("r"). The as keyword assigns the open file stream to the temporary file descriptor file.

2.2 Read the File Contents

In order to process the file contents within Python, I applied the .read() method to the file object to convert the raw file stream into an accessible string variable.

```
Step 2: Read contents with .read()

with open(import_file, "r") as file:
    ip_addresses = file.read()

print(ip_addresses)
# Output: '192.168.1.10 192.168.1.15 192.168.1.20 192.168.1.25 ...'
```

When using an open() statement with the "r" read parameter, calling .read() on the file descriptor converts the entire body of text into a single Python string. I assigned the resulting output to the variable ip_addresses. This step extracts the raw data from the filesystem into memory, setting up the data for parsing and filtering.

2.3 Convert the String into a List

In order to inspect and remove individual IP addresses dynamically, the single continuous string needed to be converted into a structured list. I used Python's .split() method to convert the ip_addresses string into a list of strings.

```
Step 3: Convert string to list with .split()

# Convert space-separated string into a list of individual IP strings
ip_addresses = ip_addresses.split()

print(ip_addresses)
# Output: ['192.168.1.10', '192.168.1.15', '192.168.1.20', '192.168.1.25']
```

The .split() function is called directly on a string object. By default, .split() segments text using whitespace (spaces, tabs, and newlines) as the delimiter, transforming a continuous block of text into distinct list elements. In this algorithm, .split() takes the string of IP addresses stored in ip_addresses and converts it into an iterable list where each item is an individual IP address. The resulting list is reassigned back to the ip_addresses variable.

2.4 Iterate Through the Remove List

A key part of the algorithm involves systematically evaluating every IP address designated for access revocation. To achieve this, I incorporated a for loop to iterate over remove_list.

```
Step 4: Iterate through remove_list with for loop

remove_list = ["192.168.1.15", "192.168.1.25", "192.168.1.40"]

# Loop through each flagged IP address in remove_list
for element in remove_list:
    # Evaluate each IP address sequentially
    print(f'Checking IP: {element}')
```

The for loop in Python repeats execution for every item within an iterable sequence. In this algorithm, the for keyword initializes the loop, assigning each revoked IP address in turn to the loop variable element. As the loop cycles through remove_list, it evaluates each unauthorized IP against the active entries in the allow list.

2.5 Remove IP Addresses That Are on the Remove List

The core filtering logic requires removing any IP address from ip_addresses that appears within remove_list. Because attempting to remove an element that does not exist in a list raises a runtime ValueError in Python, defensive validation was implemented.

```
Step 5: Remove revoked IPs using if condition and .remove()

for element in remove_list:
    if element in ip_addresses:
        ip_addresses.remove(element)

print(ip_addresses)
# Output: ['192.168.1.10', '192.168.1.20'] (Revoked IPs successfully removed)
```

Inside the body of the for loop, I implemented a conditional if statement: if element in ip_addresses:. This checks whether the current revoked IP address is indeed present in the active list. If the condition evaluates to True, ip_addresses.remove(element) is executed, safely deleting the unauthorized IP address from the list. If the IP address is not present, the loop safely skips it, ensuring the program runs without interruption.

2.6 Update the File with the Revised List of IP Addresses

As the final step, the updated, sanitized list of authorized IP addresses must be saved back to the "allow_list.txt" file. To prepare the list data for file writing, I converted the list back into a newline-separated string using the .join() method, then overwrote the file using .write().

```
Step 6: Convert to string with .join() & Overwrite with .write()

# Convert list back into a newline-separated string
ip_addresses = "\n".join(ip_addresses)

# Open allow_list.txt in write mode to overwrite with sanitized list
with open(import_file, "w") as file:
    file.write(ip_addresses)

print("allow_list.txt successfully updated.")
```

The .join() method combines all elements of an iterable into a single string, separated by the string upon which .join() was invoked. I used "\n".join(ip_addresses) to place each authorized IP address on its own line, matching enterprise access list standards.

Next, I utilized a with statement with open(import_file, "w") as file:. The "w" write mode creates a write stream that completely overwrites existing file content with the newly sanitized data. Calling file.write(ip_addresses) replaces the old access list with only authorized IPs, ensuring that revoked personnel can no longer authenticate to restricted healthcare network resources.

3. Modular Production Implementation (Refactored Function)

To make the algorithm reusable for ongoing DevOps and automated security pipelines, I encapsulated the workflow into a reusable function named update_file():

Complete Production Algorithm: update_file() Function

```python
def update_file(import_file, remove_list):
    # Open allow list file in read mode
    with open(import_file, "r") as file:
        ip_addresses = file.read()

    # Convert string to list of IP addresses
    ip_addresses = ip_addresses.split()

    # Iterate through remove_list and remove matching addresses
    for element in remove_list:
        if element in ip_addresses:
            ip_addresses.remove(element)

    # Join list into string and rewrite file
    ip_addresses = "\n".join(ip_addresses)
    with open(import_file, "w") as file:
        file.write(ip_addresses)
```

4. Project Summary & Security Value

I developed an automated Python algorithm that systematically removes revoked IP addresses from an organizational allow list file ("allow_list.txt"). The algorithm handles file I/O safely via with statements, parses text data using .read() and .split(), applies defensive conditional checks (if element in ...) inside an iterative for loop, and commits sanitized data back to disk using .join() and .write().

Security Significance: In highly regulated industries such as healthcare (HIPAA compliance), unauthorized access to electronic personal health records carries severe legal and financial repercussions. Automating the revocation of network access minimizes latency between personnel role changes and perimeter enforcement, effectively eliminating the risk of lingering unauthorized access.